Introduction au Modèle Relationnel et aux Requêtes SQL de Base (ANSI)

Thème principal : Modèle Relationnel, Clés Primaires/Étrangères, Intégrité Référentielle et Jointures Explicites (INNER JOIN)

Synthèse exécutive : Ce cours pose les fondations théoriques et pratiques de la ressource R1.05 (Introduction aux Bases de Données) en BUT 1 Informatique. Il justifie la transition nécessaire des fichiers plats et tableurs (Excel) vers les Systèmes de Gestion de Bases de Données Relationnelles (SGBD) afin d'éliminer la redondance de données et d'éviter les anomalies de mise à jour. Le cours formalise la structuration des tables à l'aide des clés primaires (unicité) et des clés étrangères (liens), garantissant la contrainte d'intégrité référentielle. Enfin, il enseigne l'écriture de requêtes SQL d'extraction fiables et normalisées (norme ISO/ANSI) privilégiant la jointure explicite INNER JOIN ON par rapport à la syntaxe implicite obsolète du produit cartésien.

1. Le Pourquoi : Des Tableurs Plats aux Bases de Données Relationnelles

Dans le développement d'applications informatiques, stocker l'information dans un fichier plat (texte, CSV) ou un tableur (Excel) montre très vite des limites critiques. Lorsqu'une donnée (comme l'adresse d'un client) est répétée sur 50 lignes de commandes différentes, modifier cette adresse exige 50 opérations distinctes. Si une seule ligne est oubliée, la base souffre d'une anomalie de mise à jour et devient incohérente. Le modèle relationnel résout ce problème à la source en découpant le système en entités distinctes (tables 'Clients', 'Commandes', etc.) reliées logiquement entre elles par des pointeurs d'identifiants.

- Tableur plat vs SGBDR : Inconvénients majeurs des tableaux plats (Excel) pour les grands volumes de données interconnectées.
- Redondance et Anomalies : La duplication des données provoque des incohérences lors des modifications, insertions ou suppressions.
- SGBD (Système de Gestion de Bases de Données) : Logiciel garantissant la cohérence, l'intégrité, la sécurité et la persistance des données.

Cas concret enseignant : *L'enseignant a pris l'exemple d'un système bancaire : si une banque enregistrait le nom, le prénom et l'adresse du titulaire sur chaque ligne de virement ou d'achat par carte bancaire, un simple déménagement nécessiterait de modifier des milliers de lignes dans l'historique des transactions passées. Cela saturerait l'espace de stockage et créerait un risque critique d'erreur manuelle.*

2. Clés Primaires, Clés Étrangères et Intégrité Référentielle

Pour garantir l'unicité des enregistrements dans une relation, on définit une clé primaire (PK). Elle doit être unique et ne jamais être nulle. Pour matérialiser une association entre deux tables (par exemple associer un Étudiant à son Groupe de TD), on n'insère pas les détails du groupe dans l'étudiant : on se contente de stocker la clé primaire du Groupe au sein de la table Étudiant en tant que Clé Étrangère (FK). Le SGBD contrôle automatiquement toute manipulation sur ces tables : il est impossible d'affecter un étudiant à un groupe non répertorié ou de supprimer un groupe sans gérer les étudiants qui y sont rattachés.

- Clé Primaire (Primary Key - PK) : Attribut (ou ensemble d'attributs) identifiant chaque ligne (n-uplet) de façon unique et obligatoirement non nulle (NOT NULL).
- Clé Étrangère (Foreign Key - FK) : Attribut dans une table réceptrice qui fait référence directe à la clé primaire d'une table parent.
- Contrainte d'Intégrité Référentielle : Règle métier automatisée par le SGBD interdisant une clé étrangère de pointer vers un identifiant parent inexistant.

Cas concret enseignant : *L'enseignant a illustré cela avec le Numéro de Sécurité Sociale (NIR) ou numéro d'étudiant INSEE : même si deux personnes portent rigoureusement les mêmes nom et prénom au Pays Basque (ex: deux Jean Dupont), leur identifiant unique empêche toute collision ou confusion au sein du système d'information de l'IUT.*

3. Extraction de Données avec SQL ANSI : SELECT, WHERE et INNER JOIN

Pour extraire des données réparties sur plusieurs tables, la norme SQL ANSI impose la syntaxe explicite via la clause INNER JOIN associée à la condition de liaison ON (ex: `FROM TableA a INNER JOIN TableB b ON a.fk_b = b.pk_b`). Cette forme est supérieure à l'ancienne syntaxe implicite du produit cartésien (`FROM TableA, TableB WHERE...`) car elle sépare clairement la logique de structure (la jointure des tables) de la logique de filtrage des données (la clause WHERE).

- Langage Déclaratif : Le SQL exprime CE QUE l'on souhaite obtenir et non les étapes de calcul (COMMENT).
- Projection (SELECT) et Filtrage (WHERE) : Définition précise des colonnes à afficher et des critères de sélection des n-uplets.
- Jointure Explicite (INNER JOIN ... ON) : Combinaison normalisée ISO/ANSI de deux tables sur la correspondance PK/FK.

Cas concret enseignant : *Avertissement strict de l'enseignant pour les contrôles et les projets : 'N'utilisez jamais la syntaxe implicite avec une virgule dans le FROM (ex: FROM Etudiant, Groupe WHERE Etudiant.id_g = Groupe.id). Utilisez TOUJOURS la syntaxe explicite INNER JOIN ... ON. Le produit cartésien implicite fait perdre des points en partiel et est formellement interdit en entreprise.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne doit-on pas utiliser SELECT * dans une application réelle ou lors d'un examen ?
Réponse du professeur : L'utilisation de 'SELECT *' est une mauvaise pratique pour trois raisons : 1) Performance : on rapatrie inutilement des colonnes inutiles sur le réseau. 2) Fragilité du code : si la structure de la table évolue (ajout/suppression de colonnes dans la BD), l'application cliente peut planter si elle s'attend à un ordre fixe de colonnes. 3) Évaluation : en examen, le professeur évalue la capacité à réaliser une projection exacte des attributs demandés par l'énoncé.

Question étudiant : Que se passe-t-il si l'on tente de supprimer un groupe qui contient encore des étudiants rattachés ?
Réponse du professeur : Par défaut, le SGBD bloque immédiatement l'opération et lève une erreur de contrainte d'intégrité référentielle (RESTRICT / NO ACTION). Tant qu'il existe des lignes filles (Étudiants) pointant vers la clé primaire de la ligne parente (Groupe), la suppression de la ligne parente est interdite à moins d'avoir configuré une suppression en cascade (ON DELETE CASCADE) ou une mise à NULL.

■ Exercice & Démonstration d'Application

Énoncé : On considère le schéma relationnel suivant : - Etudiant (id_etudiant, nom, prenom, #id_groupe) - Groupe (id_groupe, nom_groupe, promo) Écrire la requête SQL conforme à la norme ISO/ANSI permettant d'extraire le nom et le prénom des étudiants appartenant à la promotion 'BUT1-INFO' et inscrits dans le groupe nommé 'A1'.

Résolution pas à pas :
Étape 1 : Identifier les tables nécessaires et définir des alias clairs pour l'écriture -> Etudiant (e) et Groupe (g). Étape 2 : Établir la jointure explicite normalisée entre la clé étrangère de la table Etudiant et la clé primaire de la table Groupe -> INNER JOIN Groupe g ON e.id_groupe = g.id_groupe. Étape 3 : Poser les critères de filtrage dans la clause WHERE -> g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'. Étape 4 : Définir la projection explicite des attributs demandés dans le SELECT -> e.nom, e.prenom.

Résultat : SELECT e.nom, e.prenom FROM Etudiant e INNER JOIN Groupe g ON e.id_groupe = g.id_groupe WHERE g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'; Commentaire : Cette requête respecte la norme ANSI, évite le produit cartésien, et isole correctement le filtrage de la condition de jointure.

■ Auto-Évaluation Flash d'Examen

Q : Quelles sont les deux contraintes fondamentales obligatoires d'une Clé Primaire (Primary Key) ?

R : Une Clé Primaire doit obligatoirement garantir l'unicité de la valeur pour chaque n-uplet et être strictement non nulle (NOT NULL).

Q : Quelle est la différence fondamentale entre une jointure explicite (INNER JOIN ... ON) et une jointure implicite (produit cartésien dans le FROM) ?

R : La jointure explicite (INNER JOIN ... ON) isole la condition d'association des tables dans la clause ON et réserve la clause WHERE exclusivement au filtrage des données. La jointure implicite (virgule dans le FROM) réalise d'abord un produit cartésien puis filtre dans le WHERE, ce qui est moins lisible, sujet aux erreurs et non conforme aux normes modernes.

Q : À quoi sert la contrainte d'intégrité référentielle dans un SGBD relationnel ?

R : Elle garantit qu'une clé étrangère (FK) stockée dans une table pointe systématiquement vers une clé primaire (PK) existante dans la table référencée, empêchant ainsi la présence d'enregistrements 'orphelins' ou invalides.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765